

Prompt and Model Versioning

Artifact Registry · Golden-Set CI Gate · A/B Testing · Rollout Patterns · Rollback · Version Stamping

WEEK 4 · DAY 23

Tuesday, July 28 2026

Day 23 of 84

THE PRINCIPLE: a prompt is not configuration and not a string constant. It is a MODEL WEIGHT you can edit in a text editor. Day 21's ablation proved it — removing one CoT instruction cost 10.5pp of accuracy, a larger regression than most production bugs. Yet at most teams a prompt change is: edit a string, ship. No version, no review, no test, no rollback.

Closing Day 21's #1 Open Item

Day 21's design review ended with an admission: "How do you know it hasn't regressed?" — "Today, I don't." Day 22 built the detection layer that catches regressions AFTER they reach production. Today builds the gate that stops them BEFORE.

Sec	Topic	Coverage
Sec 1	Versioned Artifacts	6 artifact kinds, immutability, prompt semver
Sec 2	Prompt Registry	Immutable artifacts + movable stage pointers
Sec 3	Golden-Set CI Gate	5 checks, measure both sides, GitHub Actions wiring
Sec 4	A/B Testing	Sample size, sticky assignment, guardrails-first analysis
Sec 5	Rollout Patterns	Shadow / canary ramp / blue-green / tenant-scoped
Sec 6	Request-Path Wiring	Artifact resolution + version stamping on responses
Sec 7	FAANG Practice	Google/Microsoft/Amazon/Meta/Netflix versioning + rollout
Sec 8	Architect's Lens	Gate placement, embedder trap, rollback drill
Sec 9	Exercises + Summary	Immutability, honest gate failure, sample size, drill

Section 1 — What Is a Versioned Artifact?

Six things change your system's output. All six need versions. Most teams version one. If it can change the output and it is not in the registry, you have an unmeasured deploy vector.

[illegible]

Section 2 — The Prompt Registry

The design is deliberately borrowed from container registries: IMMUTABLE layers, MOVABLE tags. Deploying is moving a pointer. Rolling back is moving it back. Neither requires a code deploy.

```
CREATE TABLE artifacts (
    kind TEXT, name TEXT, version TEXT,
    content TEXT NOT NULL, content_hash TEXT NOT NULL,
    parent TEXT, created_at TIMESTAMPTZ DEFAULT NOW(),
    created_by TEXT NOT NULL, notes TEXT DEFAULT '',
    PRIMARY KEY (kind, name, version));

-- Stage pointers: which version is live per stage, PER TENANT.
-- tenant_id='' is the default for all tenants.
CREATE TABLE stage_pointers (
    kind TEXT, name TEXT, stage TEXT, tenant_id TEXT DEFAULT '',
    version TEXT NOT NULL, moved_at TIMESTAMPTZ DEFAULT NOW(),
    moved_by TEXT NOT NULL,
    eval_run_id TEXT,          -- the CI gate run that AUTHORIZED this move
    PRIMARY KEY (kind, name, stage, tenant_id));

-- Full audit trail. Every pointer move, forever.
CREATE TABLE pointer_history (
    id BIGSERIAL PRIMARY KEY, kind TEXT, name TEXT, stage TEXT,
    tenant_id TEXT, from_version TEXT, to_version TEXT,
    moved_at TIMESTAMPTZ DEFAULT NOW(), moved_by TEXT,
    reason TEXT, eval_run_id TEXT);

class PromptRegistry:
    async def register(self, artifact: Artifact) -> Artifact:
        """Store a new IMMUTABLE version. Rejects re-register with different
        content – that is exactly the mutation we are preventing."""
        existing = await conn.fetchrow("SELECT content_hash FROM artifacts "
            "WHERE kind=$1 AND name=$2 AND version=$3", ...)
        if existing:
            if existing["content_hash"] != artifact.content_hash:
                raise ValueError(
                    f"IMMUTABILITY VIOLATION: {artifact.ref} already exists with "
                    f"different content (stored {existing['content_hash']}, "
                    f"submitted {artifact.content_hash}). "
                    f"Bump the version instead of editing a released one.")
            return artifact          # identical re-register is a no-op
        await conn.execute("INSERT INTO artifacts (...) VALUES (...)", ...)

    async def resolve(self, kind, name, stage="production", tenant_id="") -> Artifact:
        """THE HOT PATH – called on every request.
        Tenant-specific pointer WINS over the '' default. This is what lets
        you canary a prompt for one tenant only."""
        row = await conn.fetchrow("""
            SELECT a.* FROM stage_pointers p
            JOIN artifacts a ON a.kind=p.kind AND a.name=p.name
                           AND a.version=p.version
            WHERE p.kind=$1 AND p.name=$2 AND p.stage=$3
                   AND p.tenant_id IN ($4, '')
            ORDER BY (p.tenant_id = $4) DESC    -- tenant-specific first
            LIMIT 1""", ...)
```

```
async def promote(self, kind, name, version, stage, moved_by, reason,
                  eval_run_id=None, tenant_id="") -> dict:
    """Move a stage pointer. THIS IS THE DEPLOY.

    HARD RULE: promotion to production REQUIRES a passing eval_run_id.
    This is Day 21's golden-set CI gate, enforced at the one place it
    cannot be bypassed – the promotion path itself."""
    if stage == "production" and not eval_run_id:
        raise PermissionError(
            "Promotion to production requires eval_run_id from a passing "
            "golden-set run. Untested prompt changes are unmeasured deploys.")
    # ... upsert stage_pointers + append pointer_history

async def rollback(self, kind, name, stage, moved_by, tenant_id="") -> dict:
    """Move the pointer to the previous version. Seconds, not a redeploy.
    Deliberately does NOT require an eval_run_id – you roll back TO a
    version that ALREADY passed. Requiring a gate here would mean needing
    a green build during an incident, which is exactly when you cannot
    get one."""
    prev = await conn.fetchval("SELECT from_version FROM pointer_history "
                                "WHERE ... ORDER BY moved_at DESC LIMIT 1")
    return await self.promote(kind, name, prev, stage, moved_by,
                              reason="ROLLBACK", eval_run_id="rollback-exempt")
```

Section 3 — The Golden-Set CI Gate

Day 21's #1 open item. Runs on every prompt change, before merge. CRITICAL DESIGN POINT: measure BOTH sides in the same run. Comparing against a stored baseline number is the classic mistake — that baseline was measured on different hardware, a different model version, and a different corpus.

```
@dataclass
class GateThresholds:
    """Asymmetric on purpose: blocking a regression matters more than
    celebrating an improvement."""
    max_accuracy_drop_pp: float = 2.0    # hard block below this
    max_p95_increase_pct: float = 25.0   # latency regression guard
    max_cost_increase_pct: float = 30.0   # token bloat guard
    min_faithfulness: float = 0.97       # never trade safety for accuracy
    require_no_new_class: bool = True    # no NEW failure categories

class GoldenSetGate:
    async def run(self, baseline_refs: dict, candidate_refs: dict) -> GateResult:
        run_id = f"gate-{uuid.uuid4().hex[:10]}"
        # Measure BOTH sides in the SAME run – this is the whole point
        base_rep = await CapstoneEvaluator(self.make_system(baseline_refs)).run(cases)
        cand_rep = await CapstoneEvaluator(self.make_system(candidate_refs)).run(cases)
        delta_pp = (cand_rep.execution_accuracy - base_rep.execution_accuracy)*100
        reasons = []

        # CHECK 1: accuracy regression (the primary gate)
        if delta_pp < -self.th.max_accuracy_drop_pp:
            reasons.append(f"Accuracy regression: {delta_pp:+.1f}pp "
                           f"({base_rep.execution_accuracy:.1%} -> "
                           f"{cand_rep.execution_accuracy:.1%})")

        # CHECK 2: latency regression
        p95_delta = (cand_rep.p95_latency-base_rep.p95_latency)/base_rep.p95_latency*100
        if p95_delta > self.th.max_p95_increase_pct:
            reasons.append(f"Latency regression: p95 +{p95_delta:.0f}%")

        # CHECK 3: cost regression (prompt bloat)
        base_tok = base_rep.avg_tokens_in + base_rep.avg_tokens_out
        cand_tok = cand_rep.avg_tokens_in + cand_rep.avg_tokens_out
        if (cand_tok-base_tok)/base_tok*100 > self.th.max_cost_increase_pct:
            reasons.append(f"Cost regression: tokens +{...:.0f}%")

        # CHECK 4: faithfulness floor – never trade safety for accuracy
        if cand_rep.faithful < self.th.min_faithfulness:
            reasons.append(f"Faithfulness below floor: {cand_rep.faithful:.1%}")

        # CHECK 5: no NEWLY-BROKEN cases
        # A change can hold accuracy FLAT while breaking a category that used
        # to work and fixing one that did not. Net-zero – but the newly-broken
        # category may be the one your users depend on.
        base_failed = {f["question"] for f in base_rep.failures}
        cand_failed = {f["question"] for f in cand_rep.failures}
        newly_broken = cand_failed - base_failed
        if newly_broken:
            reasons.append(f"{len(newly_broken)} previously-passing cases now fail")
```

[illegible]

Section 4 — A/B Testing Prompt Changes

The gate proves a change is not harmful on 60 curated cases. A/B testing proves it helps on REAL traffic. These answer different questions — and the statistics matter, because shipping on an underpowered test is how teams convince themselves that noise is an improvement.

```
@dataclass
class Experiment:
    name: str
    control_ref: str # "prompt/sql_system@2.3.0"
    treatment_ref: str # "prompt/sql_system@2.4.0"
    traffic_split: float = 0.10 # start at 10% treatment
    metric: str = "user_thumbs_up_rate"
    mde: float = 0.03 # minimum detectable effect (3pp)
    alpha: float = 0.05
    power: float = 0.80
    guardrail_metrics: list = field(default_factory=lambda:
        ["p95_latency_ms", "cost_per_query_usd", "faithfulness_rate"])

    def required_sample_size(self, baseline_rate: float) -> int:
        """Two-proportion sample size. Compute this BEFORE starting.
        If you cannot reach N in a reasonable window, the experiment cannot
        answer the question — do not start it and then squint at numbers."""
        z_a = stats.norm.ppf(1 - self.alpha/2); z_b = stats.norm.ppf(self.power)
        p1, p2 = baseline_rate, baseline_rate + self.mde
        p_bar = (p1 + p2)/2
        n = ((z_a*math.sqrt(2*p_bar*(1-p_bar)) +
            z_b*math.sqrt(p1*(1-p1)+p2*(1-p2)))*2) / (self.mde**2)
        return math.ceil(n) # PER ARM

class ExperimentRouter:
    def assign(self, user_id: str) -> str:
        """STICKY assignment: deterministic hash bucketing.
        Same user -> same arm, always. A user who sees two different prompt
        behaviors in one session produces noise AND a bad experience."""
        h = int(hashlib.sha256(f"{self.exp.name}:{user_id}".encode())
            .hexdigest()[:8], 16)
        return (self.exp.treatment_ref if (h % 10_000)/10_000 < self.exp.traffic_split
            else self.exp.control_ref)

class ExperimentAnalyzer:
    """THREE outcomes, not two — 'not yet significant' is valid and common."""
    def analyze(self, exp, control: ArmStats, treatment: ArmStats) -> dict:
        required_n = exp.required_sample_size(control.rate)

        # UNDERPOWERED: refuse to call it
        if control.n < required_n or treatment.n < required_n:
            return {"decision": "continue",
                "reason": f"Underpowered: {min(control.n, treatment.n):,} of "
                    f"{required_n:,} required per arm."}

        # GUARDRAILS FIRST: a win on the primary metric does NOT excuse a
        # regression on latency, cost, or faithfulness.
        breaches = []
        if treatment.p95_latency > control.p95_latency*1.25: breaches.append("p95 latency")
        if t_cost > c_cost*1.30: breaches.append("cost")
        if t_faith < 0.97: breaches.append("faithfulness")
        if breaches:
```

```

        return {"decision":"reject","reason":"Guardrail breach: "+"; ".join(breaches)}

# PRIMARY METRIC: two-proportion z-test
z, p_value = self._two_proportion_z(control, treatment)
lift_pp = (treatment.rate - control.rate)*100
if p_value >= exp.alpha:
    return {"decision":"no_difference",
            "reason":f"No significant difference (p={p_value:.3f}). "
                    f"Prefer the SIMPLER/CHEAPER prompt."}

if lift_pp < 0:
    return {"decision":"reject",
            "reason":f"Significant REGRESSION: {lift_pp:.2f}pp"}
return {"decision":"ship", "lift_pp":round(lift_pp,2),
        "next_step":"Ramp 10% -> 50% -> 100%, holding guardrails at each step."}

```

THE STATISTICAL DISCIPLINE THAT ACTUALLY MATTERS

1. COMPUTE N BEFORE YOU START. If required N is 12,000/arm and you serve 400 queries/day, the experiment takes 60 days. Decide that up front – not on day 12 when someone asks "can we ship it yet?"
2. NO PEEKING. Checking significance repeatedly and stopping at the first $p < 0.05$ inflates your false positive rate FAR above 5%. Fix N in advance, or use a sequential test designed for continuous monitoring.
3. GUARDRAILS BEAT THE PRIMARY METRIC. +2pp thumbs-up while cost +40% and faithfulness below floor is a REJECTION, not a tradeoff discussion. Encode it in code so it is not re-litigated per experiment.
4. "NO DIFFERENCE" IS A RESULT – and usually means REVERT. If a longer, more expensive prompt shows no lift, keep the shorter one.


```
# ■■■ PATTERN 4: TENANT-SCOPED ■■■
```

```
# Move the stage pointer for ONE tenant first (registry supports this  
# natively via tenant_id). An AGGREGATE canary can look healthy while one  
# tenant is completely broken.
```

```
class RolloutController:  
    """Auto-rollback on trigger breach – NO human in the loop, because the  
    breach conditions were agreed in advance and a human at 3am will  
    hesitate exactly when hesitation is most expensive."""  
    async def advance(self, plan: RolloutPlan, metrics: dict) -> dict:  
        breached = [f"{k}={metrics.get(k)} exceeds {v}"  
                     for k,v in plan.rollback_triggers.items()  
                     if metrics.get(k) is not None and metrics[k] > v]  
        if breached and plan.auto_rollback:  
            await self.registry.rollback(ArtifactKind.MODEL, ...,  
                                         stage="production", moved_by="rollout-controller")  
            return {"action": "AUTO_ROLLBACK", "breaches": breached}  
        next_step = self._next_ramp_step(plan.current_pct)  
        return {"action": "ADVANCE", "to_pct": next_step["pct"]}
```

```
# Default rollback triggers:  
# error_rate_pct 2.0 | canary_accuracy_drop_pp 2.0  
# guardrail_block_rate_pct 1.0 | cost_increase_pct 40.0
```


Section 7 — FAANG Versioning and Rollout Practice

Google — Vertex AI Model Registry and Traffic Splitting

Vertex AI provides a model registry with versioned model artifacts, and its endpoint abstraction supports deploying multiple model versions behind a single endpoint with a configurable traffic split — 95% to the current version, 5% to a candidate, then shift the percentages without changing anything client-side. The structural idea worth copying: the ENDPOINT is the stable address and the VERSION behind it is a movable pointer — exactly the stage-pointer design in Section 2. Google's SRE practice of gradual rollouts with automated rollback triggers applies unchanged to models: the ramp exists so the blast radius of a bad version is bounded by the percentage shifted so far.

Microsoft — Azure ML Registries and Prompt Flow

Azure Machine Learning provides registries as a first-class construct for sharing and versioning models, components, and environments across workspaces, and Azure AI Foundry's Prompt Flow treats a prompt-based application as a versioned, evaluable flow rather than a loose string — flows carry their own evaluation runs and can be compared across versions. The design point Microsoft makes explicit: EVALUATION RESULTS ARE ATTACHED TO THE VERSION, not stored separately. Section 3's eval_run_id on the stage pointer encodes the same relationship, so 'which eval justified this deploy?' is answerable months later rather than being an archaeology exercise in CI logs.

Amazon — SageMaker Model Registry with Approval Status

SageMaker Model Registry versions models into model package groups and attaches an explicit approval status — PendingManualApproval, Approved, or Rejected — where deployment pipelines are gated on that status. SageMaker endpoints additionally support production variants with weighted traffic and SHADOW VARIANTS that receive mirrored traffic without serving responses to users. The pattern to take is the SEPARATION OF REGISTRATION FROM APPROVAL: registering a version is cheap and unconditional; approving it for production is a distinct, auditable act with its own owner. Section 2's register() and promote() split exists for precisely this reason — anyone can register, promotion requires a passing gate.

Meta — Rollouts Gated on Guardrail Metrics

Meta's published engineering practice around experimentation emphasizes that a launch decision is evaluated against a SET of guardrail metrics, not a single primary metric — a change that improves its target while regressing a guardrail is not a tradeoff to debate but a **BLOCKED LAUNCH**. Applied to prompt and model changes, this is the discipline encoded in ExperimentAnalyzer: latency, cost, and faithfulness are checked **BEFORE** the primary metric is even considered, so a tempting accuracy win cannot quietly buy itself past a safety regression. The organizational value: the argument happens **ONCE**, when the guardrails are defined, instead of once per experiment when someone is motivated to ship.

Netflix — Sticky Assignment and Guarded Experimentation

Netflix's experimentation platform writing consistently emphasizes deterministic, **STICKY** allocation — a member's assignment must be stable so their experience is coherent and their measurements are not contaminated by switching arms mid-session — and automated analysis with checks that can **VETO** a positive result. The ExperimentRouter hash-bucketing in Section 4 implements the sticky property directly. For LLM systems the stickiness requirement is **STRONGER** than for UI experiments: a user who receives two different prompt behaviors in one conversation gets an incoherent experience they will report as a bug, which pollutes both the metrics and your incident queue.

Section 8 — Architect's Lens

THE PROMOTION PATH — where each control lives and why

```
dev          -> no gate. Anyone, anytime. Fast iteration is the point.
staging      -> smoke test only. Does it run without exploding?
canary       -> GOLDEN-SET GATE. This is the wall. Nothing untested passes.
production   -> A/B evidence OR ramp completion. Gate already passed.
```

THE GATE GOES AT canary, NOT AT production.

If the gate sits at the production boundary, teams accumulate untested changes in canary and push them all at once — the exact batch-deploy risk the gate was meant to eliminate.

WHY IMMUTABILITY IS NON-NEGOTIABLE

Mutable prompts break three things simultaneously:

1. Rollback — "revert to 2.3.0" is meaningless if 2.3.0 changed
2. Debugging — you cannot reproduce an old response
3. A/B tests — the control arm shifted mid-experiment

content_hash on register() is 5 lines and prevents all three.

VERSIONING THE EMBEDDER IS THE ONE TEAMS FORGET

Changing the embedding model invalidates EVERY vector in the index.

Retrieval silently degrades — no error, embeddings still return neighbors, they are just the WRONG neighbors. Day 22's corpus drift detector will NOT catch it either: the DDL fingerprint is unchanged.

RULE: an embedder version change REQUIRES a full re-index. Enforce it by making the embedder version PART OF THE COLLECTION NAME.

WHAT THE GATE CANNOT TELL YOU

60 golden cases prove a change is not CATASTROPHIC. They do NOT prove it HELPS real users. The gate is a FLOOR, not a ceiling.

Gate passes + A/B shows nothing -> keep the simpler prompt

Gate passes + A/B shows lift -> ship

Gate fails -> does not reach A/B at all

Teams that only run the gate ship changes that are "not harmful" forever without ever confirming any of them helped.

THE ROLLBACK DRILL NOBODY RUNS

Rollback that has never been executed is a HYPOTHESIS, not a capability.

Roll back a prompt in production ON PURPOSE, on a quiet Tuesday, and time it. If it takes more than 60 seconds it will not be used during an incident — someone will "just fix forward" and make it worse.

COST OF THIS ENTIRE DISCIPLINE

Registry: ~300 lines + 3 tables

CI gate: ~200 lines + 60 curated cases (built on Day 21/22 work)

A/B framework: ~250 lines

Ongoing: one gate run per prompt PR (~\$0.40 at Day 21 costs)

Returned: every prompt change becomes measured, reversible, and attributable. Day 21's #1 open item, CLOSED.

Section 9 — Exercises and Summary

Exercise 1: Build the Registry and Prove Immutability

```
# Register sql_system_prompt@1.0.0. Promote it to production.
# Now attempt to RE-REGISTER 1.0.0 with one word changed.
# Confirm: ValueError naming BOTH content hashes.
# Then register 1.1.0 with that change and promote WITHOUT an eval_run_id.
# Confirm: PermissionError. The gate cannot be bypassed by convenience.
```

Exercise 2: Make the Gate Fail Honestly

```
# Take your Day 18 system prompt. REMOVE the CoT instruction.
# Bump the version as a PATCH (dishonest — this IS a behavior change).
# Run GoldenSetGate. Expect: accuracy regression ~10pp, gate BLOCKS.
# The lesson: the gate is what tells you your version bump was wrong.
# Re-run with the change bumped as MINOR — the gate STILL blocks.
# Correct outcome: the change is rejected regardless of version label.
```

Exercise 3: Compute a Real Sample Size

```
# baseline thumbs-up rate = 0.72, MDE = 0.03, alpha = 0.05, power = 0.80
# Compute required_sample_size(). Divide by your actual daily volume.
# Answer HONESTLY: can you run this experiment in under a month?
# If not, raise the MDE (only detect bigger wins) or rely on the golden set
# alone. Do NOT run an underpowered test and then interpret the result.
```

Exercise 4: Run the Rollback Drill

```
# In staging: promote a deliberately-BAD prompt to production stage.
# Start a timer. Execute registry.rollback(). Stop the timer.
# Verify with pointer_history that the audit trail records BOTH moves.
# Target: under 60 seconds, no code deploy, no service restart.
# If you had to redeploy, your prompts are still baked into the image —
# that is the finding, and it is the thing to fix.
```

Summary

Concept	Key Point
Core principle	A prompt is a model weight you can edit in a text editor
Six artifacts	prompt, few-shot bank, model, embedder, retrieval cfg, guardrail cfg
Artifact immutability	frozen dataclass; content_hash is the true identity
Prompt semver	MAJOR=contract, MINOR=behavior, PATCH=no behavior change
Honest-bump test	A PATCH that moves accuracy was misclassified — CI reveals it
PromptRegistry	Immutable artifacts + movable stage pointers (container model)
Stage pointers	dev / staging / canary / production, per tenant
Deploy = pointer move	No code deploy; rollback is the same op in reverse
register() guard	Changed content on same version → IMMUTABILITY VIOLATION

promote() guard	Production promotion requires a passing eval_run_id
rollback() exemption	Gate-free by design — no green build mid-incident
GoldenSetGate	Day 21's #1 open item, closed
Measure both sides	Never compare to a stored baseline; re-run both in one job
Five gate checks	accuracy, latency, cost, faithfulness floor, newly-broken
Newly-broken check	Catches net-zero changes that break a category users need
Gate vs A/B	Gate = not-harmful on 60 cases; A/B = helpful on real traffic
required_sample_size	Compute BEFORE starting; underpowered tests are uninterpretable
Sticky assignment	Hash-bucket per user; mid-session arm switching is a bug
Guardrails first	Latency/cost/faithfulness checked before the primary metric
"No difference"	A valid result — ship the simpler, cheaper artifact
Shadow rollout	2x cost, zero risk, highest-fidelity signal
Canary ramp	1% → 5% → 25% → 50% → 100% with holds and per-step gates
Blue-green	Instant cutover and revert; 2x capacity during the window
Tenant-scoped	Aggregate canary can hide one totally-broken tenant
Auto-rollback	Triggers agreed in advance; no human hesitation at 3am
Version stamping	artifact_versions on every response — "cannot reproduce" → lookup
Embedder versioning	Changing it invalidates the index; put version in collection name
Gate placement	At the canary boundary, not the production boundary
Rollback drill	Untested rollback is a hypothesis; target under 60 seconds

NEXT: Day 24 — Cost Optimization at Scale

Prompt caching, semantic caching, request batching economics, model routing by query complexity, and building the cost model that decides which requests deserve the 70B model.